

Object Oriented Programming with Java

Week-1 Theory points

- Extension of java file is .java
- Command for compilation
javac filename.java
ex: javac hello.java
- Command for run
java filename
java hello
- java supports both interpreter and compiler.
- Java is platform independent
- Write once , run everywhere meaning java code can be run on any platform which has JVM
- JVM (Java Virtual Machine) is platform dependent.
- JIT (Just in Time) compiler is used to compile java program.
- JRE (Java Runtime Environment) = JVM+JIT
- Java provides more security with use of Access specifiers.
public , private, default, protected
- Source code(user understandable) is converted into bytecode (machine understandable code)
- Extension for source code is .java
- Extension for byte code is .class
- We cannot compile .class file.
- main() method structure:
public static void main(String args[])
{
}
- Variable syntax:
datatype variable name= value;
Ex:
int a= 8;
- for loop:
for(initialization ; condition ; increment/decrement)

```

{
}
Ex: for(int i=0; i<10;i++)
    {
    }

```

- While loop

```

While(condition)
{
    Increment/decrement;
}

```

Ex:

```

While(i<10)
{
    i++;
}

```

- DOWHILE loop:

```

do
{
}while(i>10);

```

- Class - User defined data type.

- User-defined blueprint or template used to create objects

- Object - object is a fundamental, real-world entity in an object-oriented program that has a unique state, behavior, and identity. It is an instance of a class and represents a specific occurrence in memory.

- Class syntax:

```

AccessSpecifier class class_name
{
}

```

Ex:

```

public class A
{
}

```

- Object syntax:

```

Classname objectname = new classname();

```

Ex:

```
A a = new A();
```

- Array : collection of variables with same datatypes.

➤ 1-D array:

Syntax:

```
Datatype [] arrayname = new datatype[size];
```

Ex:

```
int[] marks= new int[3];
```

➤ 2-D array:

Syntax:

```
Datatype [][] arrayname = new datatype[row size][col size];
```

Ex:

```
int[][] marks= new int[3][8];
```

- Method overloading: multiple methods with same name and different parameters and return type.

Ex:

Class A

```
{  
    Public int add()  
    {  
  
    }  
    Public int add(int a, int b)  
    {  
  
    }  
    Public double add(double a, double b, double c)  
    {  
  
    }  
}
```

```
}
```

```
}
```

- Constructor: A method without return type.
- Name of constructor is same as classname.
- Constructor is automatically called when an object is created.
- Explicit call of constructor is not possible.
- Types of constructors: 1) default 2) parameterized 3) copy

Syntax:

```
classname()
```

```
{
```

```
}
```

Ex:

Class A

```
{
```

```
    A() // default
```

```
    {
```

```
    }
```

```
    A(int a, int b) //parameterized
```

```
    {
```

```
    }
```

```
}
```